
Práctica N° 08

TRANSFORMACIÓN 2D — REFLEXIÓN

Pipeline Moderno: Vertex Shader y Fragment Shader

1. OBJETIVO.

- Entender los conceptos geométricos de la reflexión. Implementar y aplicar transformaciones de escalamiento y reflexión.

2. BASE TEÓRICA

2.1. Reflexión con respecto al eje X.

Una operación de reflexión 2D sobre el eje x es equivalente a realizar una operación de escalado no uniforme utilizando factores de 1 y -1 a lo largo del eje x y el eje y respectivamente (es decir, los valores de x permanecen sin cambios mientras se invierten los valores de y). Por lo tanto, para reflejar un punto $p_1 = [x_1, y_1]$ sobre el eje x para obtener $p_2 = [x_2, y_2]$, podemos usar:

$$x_2 = x_1, \quad y_2 = -y_1$$

En forma de matriz podemos escribir:

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} \quad (1)$$

donde Ref_x es una matriz 2×2 que representa la operación de reflexión sobre el eje x para puntos no homogéneos en el espacio 2D. En coordenadas homogéneas, para reflejar un punto 2D $p_1 = [x_1, y_1, 1]$ sobre el eje x para obtener $p_2 = [x_2, y_2, 1]$ usamos:

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} \quad (2)$$

donde Ref_x es una matriz de 3×3 que representa la operación de reflexión sobre el eje x para puntos homogéneos en espacio 2D.

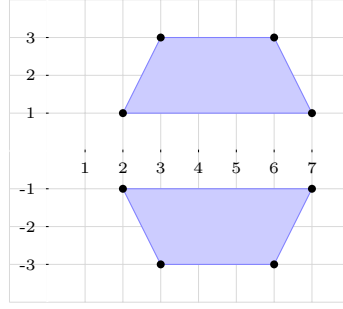


Fig. 2 Reflexión con respecto al eje X.

2.2. Reflexión con respecto al eje Y.

Una operación de reflexión 2D sobre el eje y es equivalente a realizar una operación de escalamiento no uniforme utilizando factores de -1 y 1 a lo largo del eje x y el eje y respectivamente (es decir, los valores de y permanecen sin cambios mientras se invierten los valores de x). Por lo tanto, para reflejar un punto $p_1 = [x_1, y_1]$ sobre el eje y para obtener $p_2 = [x_2, y_2]$, podemos usar:

$$x_2 = -x_1, \quad y_2 = y_1$$

Alternativamente podemos escribir:

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} -1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix} \quad (3)$$

donde Ref_y es una matriz 2×2 que representa la operación de reflexión sobre el eje y para puntos no homogéneos en el espacio 2D. En coordenadas homogéneas, usamos:

$$\begin{bmatrix} x_2 \\ y_2 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix} \quad (4)$$

donde Ref_y es una matriz 3×3 que representa la reflexión sobre el eje y para puntos homogéneos en espacio 2D.

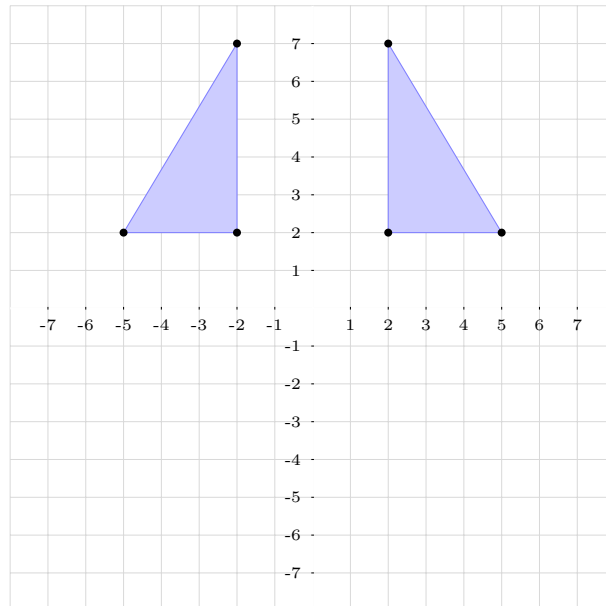


Fig. 3. Reflexión con respecto al eje Y

3. DESARROLLO DE LA PRÁCTICA (MODERN OPENGL)

A continuación, se presenta la implementación de la práctica usando **Modern OpenGL** (Vertex Shaders, Fragment Shaders) .

3.1. Shaders Base para Transformaciones

Vertex Shader (vertex_shader.glsl):

```
1 #version 330 core
2 layout (location = 0) in vec2 aPos;
3 uniform mat4 transform; // Matriz de transformacion (Escala/Reflexion/Traslacion
4 )
5 void main() {
6     // Convierte el vector 2D en homoganeo y aplica la matriz
7     gl_Position = transform * vec4(aPos, 0.0, 1.0);
8 }
```

Fragment Shader (fragment_shader.glsl):

```
1 #version 330 core
2 out vec4 FragColor;
3 uniform vec3 objectColor; // Color del objeto dibujado
4
5 void main() {
6     FragColor = vec4(objectColor, 1.0);
7 }
```

3.2. Lógica C++: Escalamiento y Reflexión

El siguiente bloque muestra cómo se implementan los cálculos matemáticos antes de enviar los datos al Vertex Shader.

```
1 #include <GL/glew.h>
2 #include <GLFW/glfw3.h>
3 #include <iostream>
4
5 // -- FUNCIONES MATEMATICAS MANUALES (Orden Column-Major) --
6
7 // Inicializa una matriz como matriz Identidad
8 void createIdentityMatrix(float* mat) {
9     for (int i = 0; i < 16; ++i) {
10         mat[i] = 0.0f;
11     }
12     mat[0] = 1.0f; // Columna 0, Fila 0
13     mat[5] = 1.0f; // Columna 1, Fila 1
14     mat[10] = 1.0f; // Columna 2, Fila 2
15     mat[15] = 1.0f; // Columna 3, Fila 3
16 }
17
18 // Crea una matriz de traslacion 2D
19 void createTranslationMatrix(float* mat, float tx, float ty) {
20     createIdentityMatrix(mat);
21     mat[12] = tx; // Columna 3, Fila 0
22     mat[13] = ty; // Columna 3, Fila 1
23 }
24
25 // Crea una matriz de escalamiento 2D
26 void createScaleMatrix(float* mat, float sx, float sy) {
27     createIdentityMatrix(mat);
28     mat[0] = sx; // Columna 0, Fila 0
29     mat[5] = sy; // Columna 1, Fila 1
30 }
```

```

30 }
31
32 // Multiplica dos matrices 4x4 (A * B) y guarda el resultado en 'result'
33 void multiplyMatrices(const float* a, const float* b, float* result) {
34     for (int col = 0; col < 4; ++col) {
35         for (int row = 0; row < 4; ++row) {
36             result[col * 4 + row] =
37                 a[0 * 4 + row] * b[col * 4 + 0] +
38                 a[1 * 4 + row] * b[col * 4 + 1] +
39                 a[2 * 4 + row] * b[col * 4 + 2] +
40                 a[3 * 4 + row] * b[col * 4 + 3];
41         }
42     }
43 }
44
45 // -- LOGICA DE TRANSFORMACIONES --
46
47 void renderTransformations(GLuint shaderProgram) {
48     glUseProgram(shaderProgram);
49     unsigned int transformLoc = glGetUniformLocation(shaderProgram, "transform")
50     ;
51
52     // --- 3.1. Escalamiento con Punto Fijo ---
53     // Punto fijo (ej. el tercer vertice del triangulo en coordenadas
54     // normalizadas)
55     float fixedX = 0.2f;
56     float fixedY = 0.2f;
57
58     // Matrices temporales
59     float matTranslateToOrigin[16];
60     float matScale[16];
61     float matTranslateBack[16];
62     float tempResult[16];
63     float finalScaleMatrix[16];
64
65     // 1. Matriz: Trasladar el punto fijo al origen (-x, -y)
66     createTranslationMatrix(matTranslateToOrigin, -fixedX, -fixedY);
67
68     // 2. Matriz: Escalar por un factor (ej. Sx = 1.5, Sy = 1.5)
69     createScaleMatrix(matScale, 1.5f, 1.5f);
70
71     // 3. Matriz: Trasladar de vuelta a la posicion original (+x, +y)
72     createTranslationMatrix(matTranslateBack, fixedX, fixedY);
73
74     // Multiplicacion: Recordar que la transformacion se aplica de derecha a
75     // izquierda:
76     // P' = T_back * Scale * T_origin * P
77     // Primero multiplicamos: Scale * T_origin
78     multiplyMatrices(matScale, matTranslateToOrigin, tempResult);
79     // Luego multiplicamos: T_back * tempResult
80     multiplyMatrices(matTranslateBack, tempResult, finalScaleMatrix);
81
82     // Enviamos la matriz manual al Shader
83     glUniformMatrix4fv(transformLoc, 1, GL_FALSE, finalScaleMatrix);
84     glDrawArrays(GL_LINE_LOOP, 0, 3);
85
86     // --- 3.2. Reflexion con respecto al eje X ---
87     float finalReflectXMatrix[16];
88     // La reflexion en X es un escalamiento de Sy = -1
89     createScaleMatrix(finalReflectXMatrix, 1.0f, -1.0f);
90
91     glUniformMatrix4fv(transformLoc, 1, GL_FALSE, finalReflectXMatrix);

```

```

90  glDrawArrays(GL_LINE_LOOP, 0, 3);
91
92
93  // --- 3.3. Reflexion con respecto al eje Y ---
94  float finalReflectYMatrix[16];
95  // La reflexion en Y es un escalamiento de Sx = -1
96  createScaleMatrix(finalReflectYMatrix, -1.0f, 1.0f);
97
98  glUniformMatrix4fv(transformLoc, 1, GL_FALSE, finalReflectYMatrix);
99  glDrawArrays(GL_LINE_LOOP, 0, 3);
100 }

```

4. ACTIVIDADES EN CLASE

- Escribir un programa que refleje un polígono de 5 lados con respecto al eje X.
- Escribir un programa para realizar reflexión general en 2D de un triángulo.

BIBLIOGRAFIA

- Donald Hearn, M. Pauline Baker (2006); Gráficas por Computadora con OpenGL; Pearson Prentice Hall; 3º edición; Madrid.
- Rimón Elias (2019); Digital Media A problem-solving Approach for Computer Graphics; eBook. New York.
- Sumanta Guha (2019); Computer Graphics Through OpenGL; Third Edition; Taylor & Francis Group.
- Gordon V. Scott, Clevenger Jhon (2019). Computer Graphics Programming in OpenGL with C++. Mercury Learning and Information.